

Propuesta de pipeline MLOps

Proyecto: Estado clínico simulado (2MLOps, Unidad 3). **Alcance académico:** no sustituye criterio médico. Este documento describe el problema, la solución propuesta y cada etapa del pipeline con argumentos de diseño y viabilidad.

0. Problema y propuesta

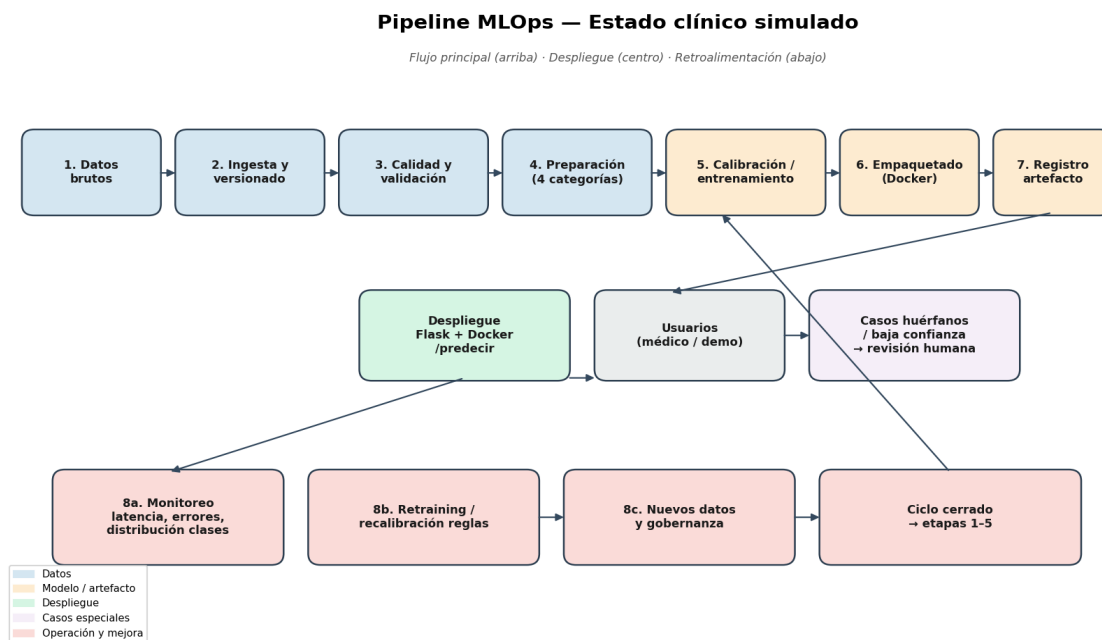
Problema. Un médico necesita una clasificación orientativa del estado de un paciente a partir de signos medibles (presión arterial, colesterol, glucosa, hábitos). El reto MLOps es entregar esa lógica de forma reproducible, versionada y desplegable.

Usuario y salida. El médico ingresa al menos tres valores vía formulario o API. El sistema devuelve una de cuatro etiquetas: *NO ENFERMO*, *ENFERMEDAD LEVE*, *ENFERMEDAD AGUDA* o *ENFERMEDAD CRÓNICA*.

Propuesta. Pipeline de extremo a extremo: datos en *data/*, etiquetado con *scripts/ajustar_cuatro_categorias.py*, lógica en *modelo_simulado.py*, servicio Flask (*POST /predecir*) e imagen Docker. Incluye camino de evolución hacia ML supervisado, monitoreo y retrabajo con datos nuevos.

Argumento MLOps. Sin pipeline, las reglas quedan aisladas en código sin trazabilidad. Con pipeline, cada versión del servicio se vincula a una versión de datos, pasa validaciones explícitas y se despliega en un entorno reproducible (Docker), con plan ante datos nuevos y degradación de rendimiento.

Diagrama general



1. Etapas del pipeline (argumento por etapa)

Etapa 1 — Datos brutos

Almacena CSV sin transformar (*data/raw/enfermedades_cardiacas.csv*, ~70 000 filas). **Por qué:** base auditable.

Artefacto: dataset crudo. **Herramientas:** Git, DVC. **Éxito:** checksum y licencia documentados.

Etapa 2 — Ingesta y versionado

Copia controlada hacia *data/processed/*. **Por qué:** saber qué datos produjeron cada versión del servicio.

Artefacto: *enfermedades_cardiacas_limpio.csv*.

Etapa 3 — Calidad y validación

Comprueba tipos, rangos (presión 50–250, colesterol/glucosa 1–3), nulos. **Por qué:** evita errores en inferencia.

Herramientas: pandas; Great Expectations en evolución.

Etapa 4 — Preparación

Genera *categoria_clinica* con reglas docentes. Distribución: LEVE 49,9 %, CRÓNICA 30,2 %, NO ENFERMO 18,1 %, AGUDA 1,9 %. **Artefacto:** *enfermedades_cardiacas_4categorias.csv*.

Etapa 5 — Calibración / entrenamiento

Actual: función determinista en *modelo_simulado.py*, alineada al script de etiquetado (baseline interpretable).

Evolución: regresión logística o Random Forest; registro en MLflow. Promoción solo si supera reglas en validación.

Etapa 6 — Empaquetado

Dockerfile fija Python 3.12 y Flask. **Por qué:** mismo resultado en cualquier PC del evaluador. **Artefacto:** imagen *estado-clinico-demo*.

Etapa 7 — Despliegue

Contenedor en puerto 5000; *GET /* (formulario) y *POST /predecir*. Probado en localhost y desde otro PC en LAN.

Etapa 8 — Monitoreo y retrabajo

Logs, latencia, distribución de clases predichas, alertas si AGUDA cae fuera del rango histórico. Triggers de recalibración: volumen de datos nuevos, caída de métricas o calendario.

2. Enfermedades huérfanas y casos especiales

2.1 Clases minoritarias (desbalance). *ENFERMEDAD AGUDA* es solo ~1,9 % del dataset. Política: (1) reglas evalúan AGUDA antes que LEVE; (2) partición estratificada en train/val/test; (3) métricas por clase (recall/F1 de AGUDA); (4) pesos de clase u oversampling en evolución ML; (5) monitoreo de frecuencia de AGUDA en producción.

2.2 Patologías raras no representadas. El CSV no cubre enfermedades huérfanas por nombre. El sistema clasifica en cuatro estados agregados, no diagnostica patologías ultra-raras. Perfiles atípicos: respuesta conservadora (LEVE) o flag de revisión humana en evolución del API. Toda decisión clínica real requiere criterio médico.

2.3 Entrada insuficiente. Menos de tres campos válidos → HTTP 400; no se emite predicción. Valores fuera de rango físico → rechazo en etapa 3.

3. Entrenamiento y calibración

Nivel actual (consigna académica). No hay ML entrenado. *ajustar_cuatro_categorias.py* calibra reglas sobre el CSV; *modelo_simulado.py* las aplica en inferencia. Ventaja: interpretable y auditable.

Reglas (prioridad): (1) AGUDA: sistólica ≥ 180 , diastólica ≥ 110 , o sistólica ≥ 160 con glucosa ≥ 3 ; (2) CRÓNICA: presencia_enfermedad=1; (3) NO ENFERMO: PA controlada, lípidos/glucosa ≤ 2 , no fumador; (4) LEVE: resto.

Evolución ML. Features tabulares → target categoria_clinica → split 70/15/15 estratificado → baseline logístico/Random Forest → métricas F1 macro y por clase → registro versionado → promoción con aprobación manual → fallback a reglas si confianza baja.

(A) Diseño

Restricciones: alcance académico, un contenedor, datos de ejemplo, sin orquestador complejo. **Limitaciones:** no uso clínico real, sesgo poblacional, cuatro categorías simplificadas — mitigadas con disclaimer, revisión humana y plan de reentrenamiento. **Datos:** CSV tabular con presión, colesterol, glucosa, fumador, presencia_enfermedad y etiqueta derivada.

(B) Desarrollo

Modelo: reglas deterministas (implementado); ML supervisado (evolución). **Pruebas:** unitarias en umbrales; contrato API JSON/HTTP; formulario web; build Docker; acceso LAN; futura matriz de confusión en test.

(C) Despliegue, monitoreo y datos futuros

Despliegue: `docker build -t estado-clinico-demo .` y `docker run -p 5000:5000`. **Monitoreo:** logs, latencia, errores 4xx/5xx, distribución de predicciones. **Datos nuevos:** ingesta versionada → validación → recalibración o reentrenamiento → evaluación → nueva imagen Docker con rollback planificado.

Conclusión de viabilidad. La propuesta es clara, las etapas son viables y están listas para ejecución en el alcance del curso: datos, scripts, servicio y Docker ya implementados; evolución ML documentada sin bloquear la entrega actual.

Documento académico — no constituye asesoría clínica.